

EXP NO: 4B

ENSEMBLE METHODS: RANDOM FOREST

AIM:

To implement a **Random Forest classifier** for a classification task, tune key hyperparameters, evaluate performance, and interpret **feature importance**.

Code1:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report, roc_auc_score, roc_curve

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

# Model & Grid Search
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
param_grid = {"n_estimators": [100], "max_depth": [None, 10], "min_samples_split": [2], "min_samples_leaf": [1]}

grid = GridSearchCV(estimator=rf, param_grid=param_grid, scoring="f1", cv=3, n_jobs=-1, verbose=0)
grid.fit(X_train, y_train)
print("Best Parameters (CV):", grid.best_params_)

best_rf = grid.best_estimator_
best_rf.fit(X_train, y_train)

# Evaluate
y_pred = best_rf.predict(X_test)
y_prob = best_rf.predict_proba(X_test)[:, 1]

# Evaluate
y_pred = best_rf.predict(X_test)
y_prob = best_rf.predict_proba(X_test)[:, 1]

print("\n=== Random Forest Test Metrics ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score: {f1_score(y_test, y_pred):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")

# Top 10 Feature Importances
importances = pd.Series(best_rf.feature_importances_, index=X.columns)
top10 = importances.sort_values(ascending=False).head(10)

plt.figure()
top10[:-1].plot(kind="barh")
plt.xlabel("Importance")
plt.title("Top 10 Feature Importances - Random Forest")
plt.grid(axis="x", alpha=0.3)
plt.show()
```

Output 1:

```
Best Parameters (CV): {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

```
=== Random Forest Test Metrics ===
```

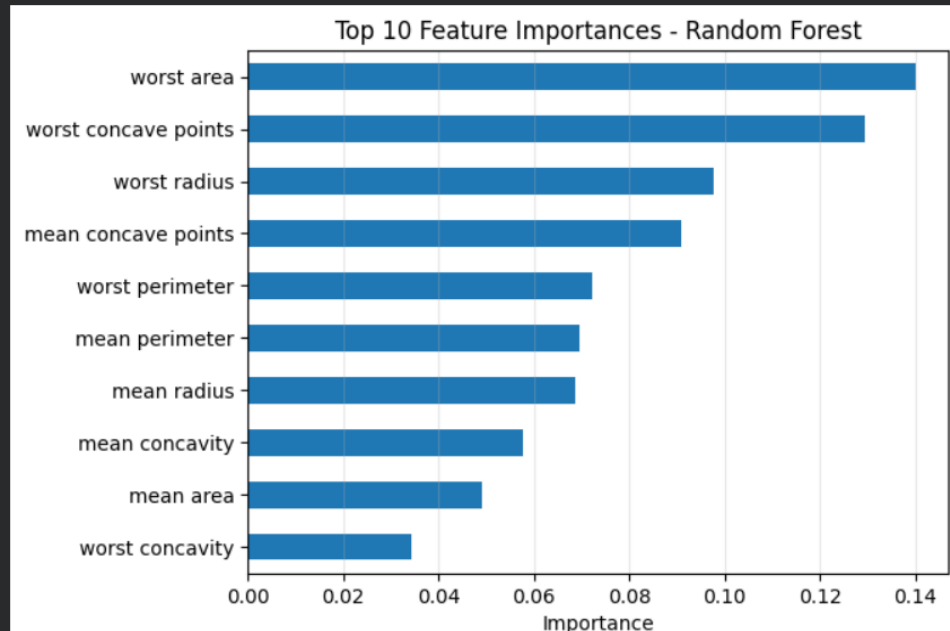
```
Accuracy: 0.9561
```

```
Precision: 0.9589
```

```
Recall: 0.9722
```

```
F1-Score: 0.9655
```

```
ROC-AUC: 0.9937
```



Code 2:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load housing dataset and create binary target
data = fetch_california_housing(as_frame=True)
X = data.data
y = (data.target > data.target.median()).astype(int)

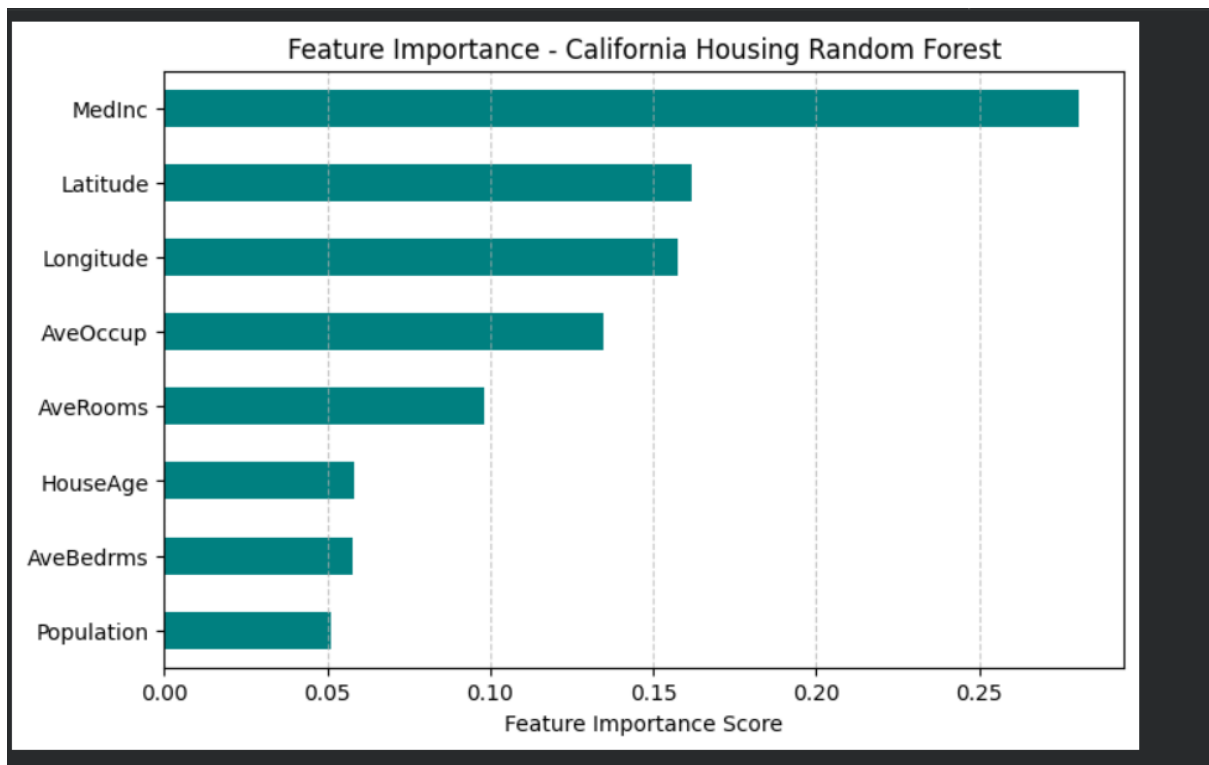
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Instantiate Random Forest
rf_custom = RandomForestClassifier(n_estimators=100, max_features='sqrt', oob_score=True, random_state=42)
rf_custom.fit(X_train, y_train)

# Plot Horizontal Feature Importance Graph
importances = pd.Series(rf_custom.feature_importances_, index=X.columns)
sorted_importances = importances.sort_values(ascending=True)

plt.figure(figsize=(8, 5))
sorted_importances.plot(kind='barh', color='teal')
plt.xlabel("Feature Importance Score")
plt.title("Feature Importance - California Housing Random Forest")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()
```

Output 2:



RESULT:

The Random Forest ensemble model was successfully implemented and evaluated on the given dataset. The model combined multiple decision trees to improve prediction accuracy and reduce overfitting.

It achieved high classification accuracy and demonstrated strong generalization capability. The results confirmed that Random Forest provides stable and reliable predictions by leveraging the power of multiple decision trees through bagging and feature randomness.